

Unit2: Array as a Data Structure

Introduction to Arrays

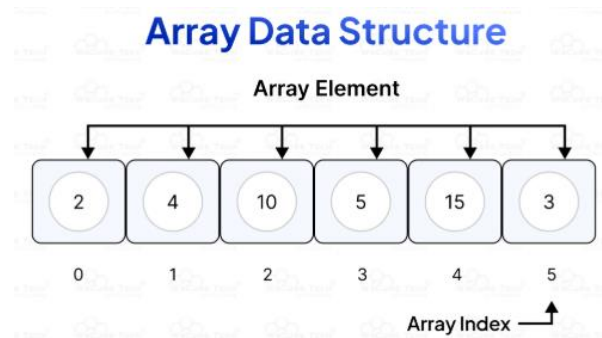
An array is a fundamental data structure used to store a collection of elements of the same data type. These elements are arranged in contiguous memory locations, allowing each element to be indexed or accessed easily and efficiently, directly using an integer index.

They are essential for various programming tasks, like data storage, retrieval, and manipulation.

Example: Arrays are like shelves in a library where each book is placed side by side in a specific order. Just as you can quickly find a book on a shelf by knowing its position, in programming, arrays let you efficiently access and manage data by its index.

Following are the important terms to understand the concept of Array –

- **Element** – Each item stored in an array is called an element.
- **Index** – Each location of an element in an array has a numerical index, which is used to identify the element.



Characteristics of Arrays in Data Structures

- **Fixed Size:** Once an array is declared, its size cannot be changed. You need to know the number of elements you will store in the array beforehand.
- **Homogeneous Elements:** All elements in an array must be of the same data type, such as all integers, all floats, or all characters.
- **Indexed by Integers:** Each element in an array is assigned a unique integer called an index, which identifies its position within the array. Indexing usually starts at 0.
- **Efficient Access:** Because of the way arrays are stored in memory (contiguously), accessing any element by its index is very efficient. This direct access using the index is often referred to as "random access."

Basics of Array in Data Structures

- **Static Arrays:**

These arrays have a fixed size that is determined at the time of creation. The size cannot be changed during the program's execution.

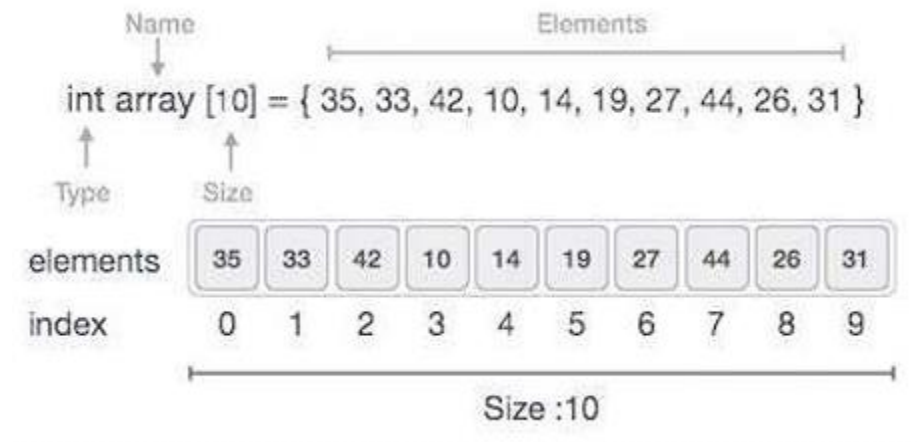
- **Dynamic Arrays:**

These arrays can grow or shrink in size as needed during the program's execution. Python lists act as dynamic arrays.

Memory Representation (Contiguous Memory Allocation)

Arrays are stored in contiguous memory locations. This means that the elements of an array are placed one after another in memory. This contiguity allows for efficient access to any element using its index. The memory representation of an array is like a long tape of bytes, with each element taking up a certain number of bytes.

For an array of type T and length N, it occupies **$N * \text{sizeof}(T)$ bytes in memory**.



From above example,

- Index starts with 0.
- Array length is 10, which means it can store 10 elements.
- Each element is accessed by using its index. For example, an element at index 6 will be fetched as 27.

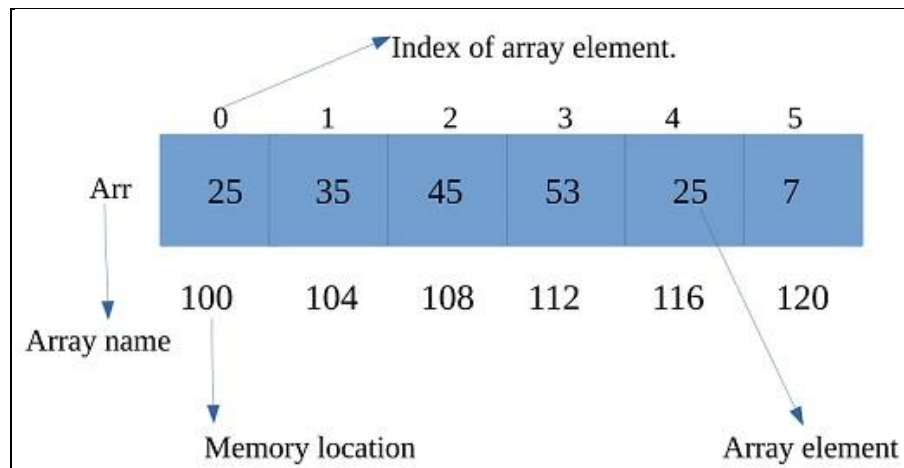
Important Points about array:

- In Python, arrays are dynamic, which means their size can be changed during runtime. Python does not have built-in static arrays like some other languages (e.g., C, C++).
- Python uses lists as its primary way to represent arrays, and lists are implemented as dynamic arrays. Dynamic arrays can grow or shrink as elements are added or removed. This is achieved by allocating a new, larger block of memory and copying the existing elements to the new location when the current capacity is exceeded.
- It's important to note that while Python lists are dynamic, the “array” module in Python provides a way to create arrays with a specific data type, which can offer some performance benefits in certain scenarios. However, these arrays are still dynamic in terms of size.
- In summary, Python arrays (implemented as lists or using the array module) are dynamic, allowing for flexible resizing during program execution.

Types of Arrays

- One dimensional array
- Two dimensional array
- Multidimensional array

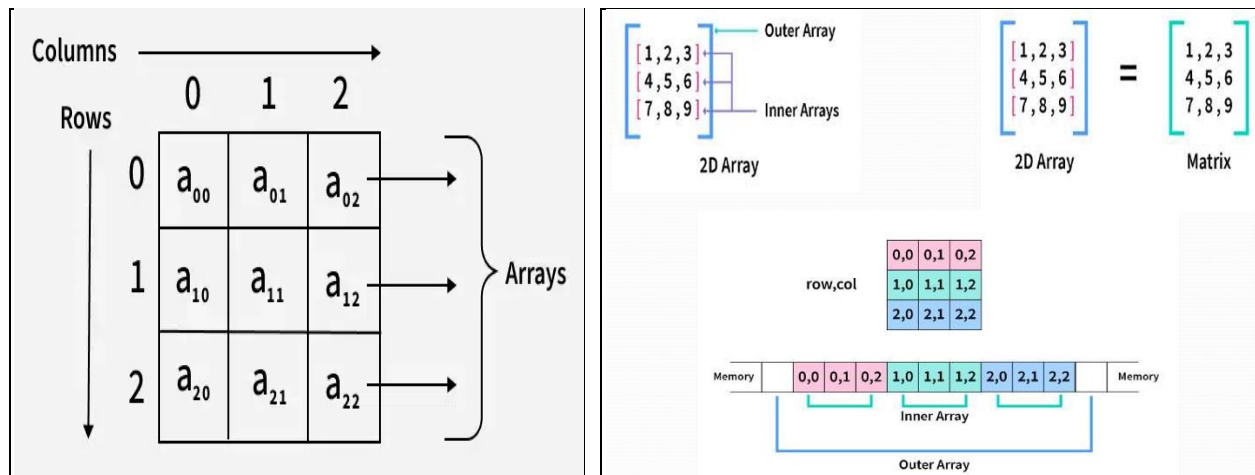
One-dimensional array contains elements only in one dimension. In other words, the shape of the NumPy array should contain only one value in the tuple. We can create a 1-D array in NumPy using the `array()` function, which converts a Python list or iterable object.



Two dimensional array (2Darray) in Python is a nested data structure, meaning it is a set of arrays inside another array. A 2D array in python is a two-dimensional data structure used for storing data generally in a tabular format.

For example, if you want to store a matrix and do some computations.

2D Array



Python Multi-dimensional Arrays

- The 2D arrays, 3D arrays etc. are called multi-dimensional arrays.
- A 2Darray contains more than 1 row and 1 column and it can be treated as a combination of several 1D arrays.
- A 2D array is also considered as a matrix. For example, a 2D array with 'm' rows and 'n' columns is called m x n matrix

Array creation:

An array is also a **linear data structure** that stores the data.

It is also **ordered, mutable, and enclosed in square brackets**.

It can store the non-unique items. But there are restrictions to store the different data type values.

To work with the Array in Python, there are 2 ways:

1. To import an array module
import array as arr
2. To import Numpy(Numerical Python)
import numpy as np

Example:

import array as arr

```
a = arr.array('i', [2, 4, 5, 6])
```

```
print(a)
```

```
print(type(a))
```

```
Output: array('i', [2, 4, 5, 6])
```

Example:

import numpy as np

```
array_sample = np.array(['samir', 'sunil', 'sachin', 'megha', 'deepti'])
```

```
print (array_sample)
```

```
print(type(array_sample))
```

```
Output: ['samir' 'sunil' 'sachin' 'megha' 'deepti']
```

Array or numpy module?

- If you're dealing with small arrays and don't need the extra functionality provided by Numpy, Python's built-in array module will be sufficient.
- However, for larger datasets and complex mathematical operations, Numpy should be used.
- While Python's built-in array module has its uses, Numpy arrays offer more functionality, speed, and efficiency.
- For data scientists dealing with large datasets and complex operations, Numpy is useful.

Creating python array Using **Import Array**:

To create an array using the 'array' module, first import the 'array' library, Then, use array.array() to create and assign a new array with specific values or no initial values:

Example:

```
import array
```

```
a = array.array('i')           # Creating an empty array of integers
```

```
a.append(1)                   # Adding elements to the array
```

```
a.append(2)
```

```
a.append(3)
```

```
print(a)
```

Importing the Array as “arr”

An array constructor is used by creating an alias for array that helps to reduce the amount of code.

So add ‘as arr’ to the import statement, replace ‘import array’ with ‘import array as arr’.

This way instead of having to type in ‘array.array()’ every time one needs a new instance created, simply typing in ‘arr.array()’ will have the same effect without needing extra words and characters typed out each time.

Example:

```
import array as arr
a = arr.array('i')          # Create an array of integers
a.append(1)                 # Add elements to the array
a.append(2)
a.append(3)
print(a)                   # Print the array
```

Output: array('i', [1, 2, 3])

Importing Array with “from array import *”

Another way to access all the functionalities of the array module is by typing ‘from array import *’. To create an array, a single line code using the constructor, ‘array()’ is enough.

Example:

```
from array import *
a = array('i')              # Creating an array
a.append(1)                 # Adding elements to the array
a.append(2)
a.append(3)
print(a)                   # Print the array
```

Output: array('i', [1, 2, 3])

Data Types in Python Arrays

The following table outlines the “typecodes” that are associated with various data types in Python when creating Python arrays:

Typecode	Value
b	Represents signed integer of size 1 byte
B	Represents unsigned integer of size 1 byte
c	Represents character of size 1 byte
i	Represents signed integer of size 2 bytes
l	Represents unsigned integer of size 2 bytes
f	Represents floating point of size 4 bytes
d	Represents floating point of size 8 bytes

Example: The below code creates an float array named numbers

```
import array as arr
numbers = arr.array('f', [15.5, 30.7, 45.9, 61.2, 75.3])
for i in numbers:
    print(i)
```

Output: 15.5 30.700000762939453 45.900001525878906 61.20000076293945
75.30000305175781

Accessing Array Elements of 1D array:

Each element of an array is accessed by using the index of the element.

Example:

```
import array as arr
a = arr.array('i', [20, 40, 50, 60])
print("First element is:", a[0])
print("Second element is:", a[1])
print("Third element is:", a[2])
print("Forth element is:", a[3])
print("Last element is:", a[-1])
print("Second last element is:", a[-2])
print("Third last element is:", a[-3])
print("Fourth last element is:", a[-4])
print(a[0], a[1], a[2], a[3], a[-1], a[-2], a[-3], a[-4])
```

Output: First element is: 20

Second element is: 40

Third element is: 50

Forth element is: 60

Last element is: 60

Second last element is: 50

Third last element is: 40

Fourth last element is: 20

20 40 50 60 60 50 40 20

Accessing Array Elements of 2D array:

1. Accessing Single Element of 2D Array

```
a=[[1,2,3],[4,5,6],[7,8,9]]
print("Element at [1][0]=",a[1][0])
Output: Element at [1][0]= 4
```

2. Accessing an Internal Array of 2D Array

```
a=[[1,2,3],[4,5,6],[7,8,9]]
print("Element at [2]=",a[2])
Output: Element at [2]= [7, 8, 9]
```

3. Display 2D Array elements

```
a=[[1,2,3],[4,5,6],[7,8,9]]
for i in a:
    for j in i:
        print(j,end=" ")
    print()
Output: 1 2 3 4 5 6 7 8 9
```

Operations on Arrays:

- **Access:** Accessing an element in an array is done using its index. The index typically starts from 0.

```
a = [1, 2, 3, 4, 5]
element = a[2]                # Accesses the element stored at index 2 = 3
```

- **Insertion:** Inserting elements into an array can be done at the end or at a specific position.

- `append()`: Adds an element to the end of the array.
- `insert()`: Adds an element at a specified index.

```
a = [1, 2, 3]
a.append(4)                # array a becomes [1, 2, 3, 4]
a.insert(1, 5)             # array a becomes [1, 5, 2, 3, 4]
```

- **Deletion:** Elements can be removed from an array by their value or index.

- `remove()`: Removes the first occurrence of a specified value.
- `pop()`: Removes an element at a specified index and returns it. If no index is specified, it removes and returns the last element.

```
a = [1, 2, 3, 2, 4]
a.remove(2)                # array a becomes [1, 3, 2, 4]
element = a.pop(1)         # element is 3, array a becomes [1, 2, 4]
element = a.pop()          # element is 4, array a becomes [1, 2]
```

- **Traversing:** Iterating through all the elements of an array.

```
a = [1, 2, 3, 4, 5]
for i in a:
    print(i)                # Prints each element of the array
```

Complexity Analysis of Operations on Arrays

- **Time Complexity:**

Operation	Best Case	Average Case	Worst Case
Traversal	$O(1)$	$O(n)$	$O(n)$
Insertion	$O(1)$	$O(n)$	$O(n)$
Deletion	$O(1)$	$O(n)$	$O(n)$
Search	$O(1)$	$O(n)$	$O(n)$
Update	$O(1)$	$O(1)$	$O(1)$

- **Space Complexity:** Most of the operations on arrays have a space complexity of $O(1)$, except for resizing operations and certain insertions/deletions that may require shifting elements, resulting in $O(n)$ space complexity.